

- Use the flags available on this scale to signal an empty repository, a non-functioning program, a norm error, cheating, etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are

profile.intra.42.fr)

objects.intra.42.fr)

arning.intra.42.fr)

network)

panies.intra.42.fr)

meta.intra.42.fr)

shop.intra.42.fr)

Menu

My projects (/)

Holy Graph (/projects/graph)

List projects (/projects/list)

Available Coursus (/cursus_subscriptions)

Your projects

Call Me Maybe (/projects/call-me-maybe)

encouraged to continue discussing your work (even if you have not finished it) in order to identify any issues that may have caused this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no unexpected, premature, or uncontrolled termination of the program is allowed; otherwise, the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explain your reasons to the evaluated learner and make sure both of you agree to this.

Attachments

 subject.pdf (<https://cdn.intra.42.fr/pdf/pdf/203343/en.subject.pdf>)

Preliminaries

Basics

- Only grade the work that is in the learner's or group's Git repository.
- Check that the following files exist and are properly named: • ex0/ft_garden_intro.py • ex1/ft_garden_data.py • ex2/ft_plant_growth.py • ex3/ft_plant_factory.py • ex4/ft_garden_security.py • ex5/ft_plant_types.py • ex6/ft_garden_analytics.py
 - Verify that no additional unauthorized files are present. If any file is missing or incorrectly named, the evaluation stops here.

 Yes

 No

General Instructions

During the evaluation of each exercise, ensure the following general instructions are met:

- The code must be written in Python 3.10 or higher.
- The code must respect the flake8 linter standards with no errors.
- A proper naming convention for functions, classes and methods is followed.
- Type hints are REQUIRED for all functions and methods (parameters and return values). Check this using mypy.
- Docstrings are NOT required for this module.
- The learner does not need to handle input validation unless explicitly mentioned.
- All programs must run without errors or crashes.

If any of these general instructions are not met, the evaluation must stop.

 Yes

 No

Exercises

Exercise 0 - Planting Your First Seed (Program Structure)

Test the ft_garden_intro.py program:

Basic functionality:

- Run the program: python ex0/ft_garden_intro.py
- Verify the program uses the if __name__ == "__main__": pattern
- Check that plant information is stored in simple variables

profile.intra.42.fr)

objects.intra.42.fr)

arning.intra.42.fr)

network)

meta.intra.42.fr)

shop.intra.42.fr)

Menu

My projects (/)

Holy Graph (/projects/graph)

List projects (/projects/list)

Available Coursus (/cursus_subscriptions)

Your projects

Call Me Maybe (/projects/call-me-maybe)

- Verify the program displays information using print()
- Ask the learner to explain what `if __name__ == "__main__":` means
- Ask the learner why this pattern is important in Python

Key concepts to verify:

- Correct use of `if __name__ == "__main__":`
- Variables to store data (name, height, age)
- Print statements to display information

Understanding check:

- Ask: "What could happen if you remove the `if __name__ == "__main__":` line?"
- Ask: "When would this code NOT run?"
- Verify they understand this is the program's entry point

Edge cases:

- Output is readable and well-formatted
- Code structure is clean and simple
- Ask the learner to launch the program with and without a shebang

Does the program demonstrate an understanding of basic Python program structure?

☒

Yes

☐

No

Exercise 1 - Garden Data Organizer (Classes)

Test the `ft_garden_data.py` program:

Basic functionality:

- Run the program: `python ex1/ft_garden_data.py`
- Verify the program defines and uses a CLASS structure
- Check that plant data is organized using class instances
- Verify the program displays plant information correctly
- Ask the learner to explain what a class is and why they used it
- Ask the learner to show where they defined the class and created instances

Key concepts to verify:

- Class definition with proper syntax
- Object instantiation (creating instances)
- Basic attributes (plant properties)
- Simple display of plant data

Edge cases:

- Output is readable and well-formatted
- Class structure is logical and appropriate

Does the program demonstrate an understanding of classes as data organization tools?

☒

Yes

☐

No

Exercise 2 - Plant Growth Simulator (Methods)

Test the `ft_plant_growth.py` program:

Basic functionality:

- Run the program: `python ex2/ft_plant_growth.py`
- Verify the program uses METHODS to perform actions on plants
- Check that plants can grow and age over time
- Verify the simulation shows changes over multiple days
- Ask the learner to explain what methods are and why they used them
- Ask the learner to demonstrate calling methods on plant objects



objects.intra.42.fr)



owteams.com/c/42network)



- Protected attributes (using underscore convention)
- Getter and setter methods for controlled access
- Data validation before allowing changes
- Error handling for invalid operations



projects.intra.42.fr



owteams.com/c/42network)



meta.intra.42.fr)



- Valid data updates work correctly
- Invalid data is rejected with appropriate messages
- Direct attribute access is discouraged
- Plant data integrity is maintained

☒ Yes

☒ No

Test the `ft_plant_types.py` program:

- Run the program: `python ex5/ft_plant_types.py`
- Verify the program uses INHERITANCE to create specialized plant types
- Check that different plant types (Flower, Tree, Vegetable) exist and have the Plant class as their parent
- Verify each type has both common and specialized features
- Ask the learner to explain inheritance and why they used it
- Ask the learner to show the parent-child relationships between classes

- Base class (Plant) with common features
- Derived classes (Flower, Tree, Vegetable) with specialized features
- `super()` calls to parent class methods
- Method overriding for specialized behavior (at least the `show()` method that can call `super().show()` eventually, or completely override it)

- All plant types share common functionality
- Each type has unique features (blooming, shade, nutrition)
- Inheritance chain works correctly
- Specialized methods function as expected

☒ Yes

☒ No

Test the `ft_garden_analytics.py` program:

- Run the program: `python ex6/ft_garden_analytics.py`
- Verify the program demonstrates ADVANCED OOP concepts
- Check for nested classes, inheritance chains, static methods, class methods
- Verify distinction between member and non-member functions
- Verify that the standalone function that displays statistics works for every Plant: it should get a Plant type as parameter and call the display function in the nested class that holds statistics.
- Ask the learner to explain the key concepts (below) that they implemented

- Nested classes (class within a class)
- Inheritance chains (A -> B -> C)
- Class methods (using `classmethod()` or the decorator) vs instance methods
- Static methods (using `staticmethod()` or the decorator) vs regular functions
- Non-member functions (outside any class)
- Nested class and methods override through inheritance (nested class N in B overrides N in A)

- Nested class functionality works correctly
- Inheritance chain demonstrates proper `super()` usage

